

Core Patterns Checklist

1. Hashing & Hash Table

Focus: Fast $O(1)$ lookups, frequency counting, grouping, and prefix sum hash maps.

- [1. Two Sum](#) (Easy) — Instant complement lookup using HashMap.
- [217. Contains Duplicate](#) (Easy) — Basic HashSet usage for dynamic duplicate detection.
- [242. Valid Anagram](#) (Easy) — Frequency map array / Hash Map counting.
- [383. Ransom Note](#) (Easy) — Constructing a frequency map to verify character availability.
- [387. First Unique Character in a String](#) (Easy) — Two-pass string traversal with a frequency map.
- [205. Isomorphic Strings](#) (Easy) — Two-way character mapping using HashMaps.
- [290. Word Pattern](#) (Easy) — String tokenization and bijection mapping.
- [771. Jewels and Stones](#) (Easy) — HashSet for $O(1)$ lookups of valid characters.
- [349. Intersection of Two Arrays](#) (Easy) — Using HashSets to find common elements uniquely.
- [169. Majority Element](#) (Easy) — Frequency counting to find the dominant element.
- [1207. Unique Number of Occurrences](#) (Easy) — Map for frequencies, Set to check uniqueness.
- [49. Group Anagrams](#) (Medium) — Key transformation & string encoding as map keys.
- [128. Longest Consecutive Sequence](#) (Medium) — HashSet for $O(N)$ sequence checking without sorting.
- [380. Insert Delete GetRandom O\(1\)](#) (Medium) — Combining a HashMap with a dynamic array for $O(1)$ operations.

2. Two Pointers

Focus: Array partitioning, opposite-end shrinking windows, and fast-and-slow traversal.

- [26. Remove Duplicates from Sorted Array](#) (Easy) — Fast & slow pointers for in-place array compression.
- [283. Move Zeroes](#) (Easy) — In-place read-write pointers while maintaining relative order.
- [125. Valid Palindrome](#) (Easy) — Opposite-end pointers with character filtering.
- [345. Reverse Vowels of a String](#) (Easy) — Conditional opposite-end pointers.
- [680. Valid Palindrome II](#) (Easy) — Opposite-end pointers with a one-time skip tolerance.
- [844. Backspace String Compare](#) (Easy) — Reverse two-pointer traversal handling skip logic.
- [167. Two Sum II - Input Array Is Sorted](#) (Medium) — Opposite-end pointers taking advantage of sorted order.
- [15. 3Sum](#) (Medium) — Sorting + Two Pointers with duplicate avoidance logic.
- [18. 4Sum](#) (Medium) — Extending the 3Sum logic with an outer loop.
- [11. Container With Most Water](#) (Medium) — Greedy opposite-end pointer movement.
- [75. Sort Colors](#) (Medium) — Dutch National Flag 3-pointer partitioning algorithm.
- [881. Boats to Save People](#) (Medium) — Sorting + greedy pairing using opposite ends.
- [42. Trapping Rain Water](#) (Hard) — A classic OA staple using a two-pointer boundary approach.

3. Sorting & Custom Comparators

Focus: Sorting as a preprocessing step, interval merging, and custom comparator logic.

- [88. Merge Sorted Array](#) (Easy) — Backward two-pointer merging to avoid extra space.

- [628. Maximum Product of Three Numbers](#) (Easy) — *Sorting to expose extremes (handling negative number edge cases).*
- [1356. Sort Integers by The Number of 1 Bits](#) (Easy) — *Custom comparator based on bit counts.*
- [1636. Sort Array by Increasing Frequency](#) (Easy) — *Custom sorting with secondary tie-breaker criteria.*
- [56. Merge Intervals](#) (Medium) — *Sorting intervals by start time and merging overlapping ranges.*
- [57. Insert Interval](#) (Medium) — *Linear traversal with interval merging logic.*
- [435. Non-overlapping Intervals](#) (Medium) — *Greedy sorting by end-time to minimize deletions.*
- [451. Sort Characters By Frequency](#) (Medium) — *HashMap frequency counting followed by sorting.*
- [179. Largest Number](#) (Medium) — *Custom string combination comparator sorting.*
- [973. K Closest Points to Origin](#) (Medium) — *Custom distance sorting.*
- [1877. Minimize Maximum Pair Sum in Array](#) (Medium) — *Sorting to optimally pair extremes.*
- [1029. Two City Scheduling](#) (Medium) — *Greedy sorting based on cost differentials.*